

Desenvolvimento Web com .NET

Aula 2

Prof. Me. Lucas Teodoro dos Santos

UNIP

19 de Agosto de 2026

Roteiro da aula

- 1 Program.cs
- 2 Middleware
- 3 Rotas
- 4 Tag Helpers
- 5 Comunicação entre as Camadas do MVC

O que é o Program.cs?

O arquivo `Program.cs` é o ponto de entrada de uma aplicação ASP.NET Core. Ele concentra as configurações necessárias para preparar, construir e iniciar a aplicação.

De maneira geral, esse arquivo é responsável por:

- Carregar as configurações iniciais da aplicação;
- Registrar os serviços que serão utilizados;
- Construir a aplicação;
- Configurar o pipeline de requisições HTTPs;
- Definir os recursos e as rotas disponíveis;
- Iniciar a execução da aplicação.

Organização

O `Program.cs` pode ser compreendido em três partes: **preparação, configuração e execução.**

Entendendo o Program.cs

```
1 var builder = WebApplication.CreateBuilder(args);
```

- Cria o objeto responsável pela configuração inicial da aplicação.
- Nessa etapa, são carregadas as configurações do projeto, preparado o sistema de logs e inicializado o container de injeção de dependência.

```
1 builder.Services.AddControllersWithViews();
```

- Registra os serviços necessários para trabalhar com Controllers e Views no padrão MVC.
- Outros serviços, como acesso ao banco de dados e autenticação, também podem ser registrados nessa etapa.

Entendendo o Program.cs

```
1 var app = builder.Build();
```

- Constrói a aplicação utilizando as configurações e os serviços registrados anteriormente.
- O objeto app representa a aplicação web que será configurada e executada.

```
1 if (!app.Environment.IsDevelopment())  
2 {  
3     app.UseExceptionHandler("/Home/Error");  
4     app.UseHsts();  
5 }
```

- Verifica se a aplicação não está sendo executada no ambiente de desenvolvimento.
- Configura o tratamento de erros e o uso de conexões HTTPS no ambiente de produção.

Entendendo o Program.cs

```
1 app.UseHttpsRedirection();
```

- Redireciona automaticamente as requisições HTTP para HTTPS.

```
1 app.UseRouting();
```

- Adiciona o sistema de roteamento ao pipeline da aplicação.
- O roteamento identifica qual recurso deve atender à URL acessada.

```
1 app.UseAuthorization();
```

- Adiciona a verificação das permissões de acesso ao pipeline.
- Quando houver controle de acesso, verifica se o usuário possui permissão para acessar o recurso solicitado.

Entendendo o Program.cs

```
1 app.MapStaticAssets();
```

- Mapeia os arquivos estáticos utilizados pela aplicação.
- Entre esses arquivos estão CSS, JavaScript, imagens e outros recursos da pasta `wwwroot`.

```
1 app.MapControllerRoute(  
2     name: "default",  
3     pattern: "{controller=Home}/{action=Index}/{id?}")  
4     .WithStaticAssets();
```

- Define a rota convencional utilizada pelos Controllers da aplicação.
- A URL pode indicar um Controller, uma Action e um identificador opcional.

Entendendo o Program.cs

```
1 app.Run();
```

- Inicia a execução da aplicação.
- A partir desse momento, a aplicação passa a aguardar e processar as requisições HTTP/HTTPS.

O que é Middleware?

Um *middleware* é um componente que participa do processamento das requisições e respostas HTTP/HTTPS.

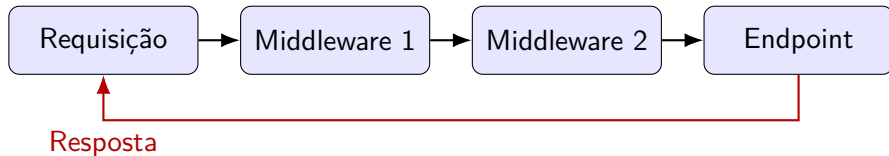
O conjunto de middlewares configurados na aplicação forma o *pipeline* de requisição.

Cada middleware pode:

- Executar uma ação antes de encaminhar a requisição;
- Encaminhar a requisição para o próximo componente;
- Executar uma ação durante o retorno da resposta;
- Interromper o pipeline e produzir diretamente uma resposta.

Pipeline de Middlewares

Os middlewares são executados na ordem em que foram adicionados ao `Program.cs`.



A ordem importa

Um middleware pode depender de informações preparadas por outro middleware executado anteriormente.

Principais Middlewares do Projeto Inicial

```
1  if (!app.Environment.IsDevelopment())
2  {
3      app.UseExceptionHandler("/Home/Error");
4      app.UseHsts();
5  }
6  app.UseHttpsRedirection();
7  app.UseRouting();
8  app.UseAuthorization();
```

Middleware	Responsabilidade
UseExceptionHandler()	Direciona os erros não tratados para uma página específica.
UseHsts()	Orienta o navegador a utilizar HTTPS nas próximas conexões.
UseHttpsRedirection()	Redireciona as requisições HTTP para HTTPS.
UseRouting()	Identifica o endpoint correspondente à URL acessada.
UseAuthorization()	Verifica se o usuário possui permissão para acessar o recurso.

Exemplo Prático de Middleware

Um exemplo comum é a criação de um middleware para registrar informações sobre todas as requisições recebidas pela aplicação.

Esse middleware pode registrar:

- A URL acessada;
- O método da requisição, como GET ou POST;
- O horário em que a requisição foi recebida;
- O tempo de processamento;
- O código de status da resposta.

Como o middleware faz parte do pipeline, esse registro pode ser realizado automaticamente para todas as requisições, sem repetir o mesmo código em cada Controller.

Exemplo de registro

GET /Produtos - Status: 200 - Tempo: 125 ms

O que são rotas?

As rotas determinam como uma URL será associada a um recurso da aplicação.

Em uma aplicação MVC, a rota normalmente indica:

- O Controller que receberá a requisição;
- A Action que será executada;
- Um possível parâmetro.

Exemplo

`/Produtos/Detalhes/5`

Rota Conventional do MVC

```
1 app.MapControllerRoute(  
2   name: "default",  
3   pattern: "{controller=Home}/{action=Index}/{id?}")  
4   .WithStaticAssets();;
```

- controller=Home: define o Controller padrão;
- action=Index: define a Action padrão;
- id?: representa um identificador opcional.

Essa rota é genérica e pode ser utilizada por diferentes Controllers.

Exemplos

/Produtos

/Produtos/Detalhes/5

Rota Personalizada

Uma rota personalizada pode ser criada para deixar uma URL mais curta, amigável ou adequada ao endereço desejado para determinado recurso.

```
1 app.MapControllerRoute(  
2     name: "catalogo",  
3     pattern: "catalogo/{controller=Produtos}/{action=Index}/{id?}")  
4     .WithStaticAssets();
```

Nesse exemplo, a palavra `catalogo` na URL direciona a requisição para o `ProdutosController`.

O que são Tag Helpers?

Os *Tag Helpers* são recursos do ASP.NET Core utilizados nas Views Razor para adicionar comportamentos do servidor aos elementos HTML.

Eles são identificados por atributos iniciados por `asp-`.

- Facilitam a criação de links e formulários;
- Integram elementos HTML aos Controllers e Actions;
- Permitem trabalhar com propriedades do Model;
- Geram automaticamente partes do código HTML.

Exemplos

`asp-controller`, `asp-action`, `asp-route-id` e `asp-for`

Tag Helpers em Links

Os atributos `asp-controller` e `asp-action` permitem criar um link para uma Action de um Controller.

Código na View Razor:

```
1 <a asp-controller="Produtos" asp-action="Index">Produtos</a>
```

HTML gerado:

```
1 <a href="/Produtos">Produtos</a>
```

O ASP.NET Core utiliza as rotas da aplicação para gerar o atributo `href`.

Enviando Parâmetros pela Rota

O atributo `asp-route-id` adiciona um identificador ao endereço gerado.

Código na View Razor:

```
1 <a asp-controller="Produtos" asp-action="Detalhes" asp-route-id="5">Ver detalhes</a>
```

HTML gerado:

```
1 <a href="/Produtos/Detalhes/5">Ver detalhes</a>
```

O valor 5 será enviado para o parâmetro `id` da Action.

Tag Helpers em Formulários

Os Tag Helpers também podem definir para onde um formulário será enviado.

Código na View Razor:

```
1 <form asp-controller="Produtos" asp-action="Cadastrar" method="post">
2     <button type="submit">Salvar</button>
3 </form>
```

HTML gerado:

```
1 <form action="/Produtos/Cadastrar"method="post">
2     <button type="submit">Salvar</button>
3 </form>
```

Tag Helper asp-for

O asp-for associa um elemento HTML a uma propriedade do Model.

Model utilizado pela View:

```
1 public class Medico
2 {
3     public string Nome { get; set; } = string.Empty;
4
5     public int Idade { get; set; }
6
7     public DateTime DataNascimento { get; set; }
8 }
```

Campos criados na View:

```
1 @model ProjetoAgendamento.Models.Medico
2
3 <input asp-for="Nome" />
4 <input asp-for="Idade" />
5 <input asp-for="DataNascimento" />
```

O ASP.NET Core analisa cada propriedade para configurar o campo correspondente.

HTML Gerado pelo asp-for

Com base nas características das propriedades, o ASP.NET Core gera aproximadamente o seguinte HTML:

```
1 <input type="text" id="Nome" name="Nome" value="" />
2
3 <input type="number" id="Idade" name="Idade" value="0" />
4
5 <input type="datetime-local" id="DataNascimento" name="DataNascimento" />
```

- O nome da propriedade é utilizado nos atributos id e name;
- O tipo da propriedade ajuda a definir o tipo do campo HTML;
- O valor atual da propriedade é colocado no atributo value;
- As Data Annotations também podem influenciar o elemento gerado.

Principais Tag Helpers

Entre os Tag Helpers mais utilizados em aplicações MVC estão:

- `asp-controller`: define o Controller de destino;
- `asp-action`: define a Action que será executada;
- `asp-route-id`: envia um identificador pela rota;
- `asp-for`: associa um elemento a uma propriedade do Model;
- `asp-validation-for`: apresenta a mensagem de validação de uma propriedade;
- `asp-validation-summary`: apresenta um resumo dos erros de validação;
- `asp-area`: define a área da aplicação utilizada na navegação.

Habilitação dos Tag Helpers

Os Tag Helpers normalmente são habilitados no arquivo `_ViewImports.cshtml`.

```
1 @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Essa instrução disponibiliza os Tag Helpers padrão do ASP.NET Core para as Views da aplicação.

Projeto MVC

Nos projetos criados pelo modelo padrão, essa configuração normalmente já está presente.

Comunicação entre as Camadas do MVC

No padrão MVC, o Controller pode utilizar objetos do Model e enviá-los para uma View responsável pela apresentação dos dados.

Nesta prática, realizaremos as seguintes etapas:

- 1 Criar o Model Medico;
- 2 Criar uma lista de médicos no Controller;
- 3 Enviar a lista para a View;
- 4 Apresentar os médicos utilizando Razor.

Fluxo da prática

Model → Controller → View

Criando o Model Médico

Na pasta Models, crie a classe Medico:

```
1 namespace ProjetoAgendamento.Models;
2
3 public class Medico
4 {
5     public int Id { get; set; }
6
7     public string Nome { get; set; } = string.Empty;
8
9     public string Crm { get; set; } = string.Empty;
10
11     public string Especialidade { get; set; } = string.Empty;
12 }
```

Essa classe representa os dados de um médico na aplicação.

Criando o MedicosController

Na pasta Controllers, crie o Controller MedicosController:

```
1 using Microsoft.AspNetCore.Mvc;  
2 using ProjetoAgendamento.Models;  
3  
4 namespace ProjetoAgendamento.Controllers;  
5  
6 public class MedicosController : Controller  
7 {  
8     public IActionResult Index()  
9     {  
10         return View();  
11     }  
12 }
```

- O Controller herda da classe Controller;
- A Action Index() atenderá a rota /Medicos;
- O método View() será responsável por carregar a View correspondente.

Criando a Lista de Médicos

Dentro da Action Index, crie os objetos que representarão os médicos:

```
1 Medico medico1 = new Medico
2 {
3     Id = 1,
4     Nome = "Ana Souza",
5     Crm = "12345",
6     Especialidade = "Cardiologia"
7 };
8
9 Medico medico2 = new Medico
10 {
11     Id = 2,
12     Nome = "Carlos Lima",
13     Crm = "67890",
14     Especialidade = "Ortopedia"
15 };
```

Adicionando os Médicos à Lista

Depois, crie a lista, adicione os objetos e envie os dados para a View:

```
1 List<Medico> medicos = new List<Medico>();  
2  
3 medicos.Add(medico1);  
4 medicos.Add(medico2);  
5  
6 return View(medicos);
```

- Os objetos `medico1` e `medico2` são adicionados à lista;
- O método `View()` envia a lista criada no Controller para a View relacionada à Action `Index`.

Criando a View de Médicos

Para criar a View responsável pela apresentação dos médicos:

- Na pasta Views, deverá ser criada uma nova pasta chamada Medicos;
- Dentro da pasta Medicos, deverá ser criado o arquivo `Index.cshtml`.

Estrutura criada

`Views/Medicos/Index.cshtml`

Recebendo os Dados na View

No início do arquivo Views/Medicos/Index.cshtml, informe o tipo de objeto recebido:

```
1 @model List<ProjetoAgendamento.Models.Medico>
```

A diretiva `@model` indica que a View receberá uma lista de objetos da classe `Medico`.

O namespace completo `NomeProjeto.Models.Medico` informa exatamente onde a classe está localizada no projeto.

A propriedade `Model` representa a lista enviada pelo Controller e pode ser percorrida utilizando o comando `foreach`.

Apresentando a Lista de Médicos (1/2)

```
1 @model List<ProjetoAgendamento.Models.Medico>
2
3 <h1>Medicos</h1>
4
5 <table class="table">
6     <thead>
7         <tr>
8             <th>Nome</th>
9             <th>CRM</th>
10            <th>Especialidade</th>
11        </tr>
12    </thead>
```

Nessa primeira parte, é criado o título da página e o cabeçalho da tabela.

Apresentando a Lista de Médicos (2/2)

```
1      <tbody>
2          @foreach (var medico in Model)
3          {
4              <tr>
5                  <td>@medico.Nome</td>
6                  <td>@medico.Crm</td>
7                  <td>@medico.Especialidade</td>
8              </tr>
9          }
10     </tbody>
11 </table>
```

O comando `foreach` percorre a lista recebida e cria uma linha da tabela para cada médico.

Adicionando o Item Médicos ao Layout

No arquivo `_Layout.cshtml`, adicione um novo item ao menu de navegação:

```
1 <li class="nav-item">
2   <a class="nav-link text-dark" asp-controller="Medicos" asp-action="Index">Medicos</a>
3 </li>
```

- `asp-controller="Medicos"` direciona para o `MedicosController`;
- `asp-action="Index"` define a Action que será executada;
- O Tag Helper gera automaticamente o endereço do link.

Resultado

Ao selecionar o item **Médicos**, a aplicação acessará a página `/Medicos`.

Nesta aula, vimos como uma aplicação ASP.NET Core MVC é configurada e como ocorre a comunicação entre seus componentes.

- Entendemos o papel do `Program.cs` na configuração e inicialização da aplicação;
- Conhecemos o conceito de **Middleware** e o funcionamento do pipeline de requisições;
- Vimos como as **rotas** relacionam URLs aos Controllers e Actions;
- Utilizamos **Tag Helpers** para integrar elementos HTML com recursos do ASP.NET Core;
- Criamos um **Model** e enviamos uma lista de objetos do Controller para a View;
- Utilizamos **Razor** para percorrer e apresentar os dados recebidos pela View.

Obrigado!

Dúvidas?